



Carrera:

Ing. Tic 's.

Materia:

Pruebas de Software

Semestre:

9º

Docente:

Oscar Lenin Espinoza Alvarez

Alumnos:

Ángel Isaac Fonseca Gómez

Carretera a la Estación de Rincón Km. 1 C.P. 20670 Pabellón de Arteaga, Aguascalientes

Tel. 465 958-2482 Ext. 104 e-mail: plan_parteaga@tecnm.mx tecnm.mx |

pabellon.tecnm.mx



Referencias:

- **Jest (Framework de pruebas unitarias):** Jest es un framework de pruebas muy popular para JavaScript, especialmente útil en proyectos Node.js con TypeScript. Permite realizar pruebas unitarias con características como mocks, aserciones y reportes de cobertura. La configuración de Jest es sencilla y se integra bien para validar la lógica del backend. La documentación oficial de Jest proporciona guías completas para su instalación y uso

<https://jestjs.io/>

- **Supertest (Biblioteca para pruebas de API):** Supertest se utiliza para realizar pruebas a APIs HTTP simulando solicitudes a los endpoints y verificando sus respuestas. Es muy útil cuando se combina con Jest para probar APIs RESTful, ya que permite realizar solicitudes de diferentes métodos HTTP y verificar que las respuestas sean correctas. La documentación oficial de Supertest ofrece ejemplos y detalles sobre cómo implementarlo en tus pruebas Documentación de Supertest

<https://github.com/visionmedia/supertest>

- **Selenium WebDriver (Herramienta para pruebas de integración):** Selenium es una herramienta ampliamente usada para la automatización de navegadores, ideal para realizar pruebas de integración de aplicaciones web. En el caso de aplicaciones Vue.js, Selenium WebDriver se puede usar para simular interacciones de usuario, como hacer clic en botones o llenar formularios, en navegadores como Firefox. Es útil para probar la funcionalidad y la usabilidad de la aplicación web de extremo a extremo

<https://selenium.dev/documentation/en>

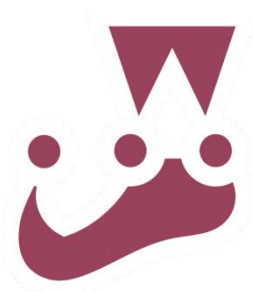
Reporte de Pruebas de Software

1. Introducción

Este reporte documenta el proceso y los resultados de las pruebas de software realizadas en el sistema de gestión de reportes de conducta de una institución. El objetivo principal de estas pruebas es asegurar la calidad, funcionalidad y rendimiento del sistema, garantizando que cumpla con los requisitos establecidos y proporcione una experiencia de usuario óptima.

Para llevar a cabo estas pruebas, se utilizaron diversas herramientas y tecnologías, cada una seleccionada por su capacidad para poder abarcar todos los campos y tipos de pruebas necesarias. Las herramientas utilizadas incluyen:

- Jest: **Jest (Framework de pruebas unitarias)**
- Supertest: **Supertest (Biblioteca para pruebas de API)**
- Selenium WebDriver **(Herramienta para pruebas de integración)**



A lo largo de este informe, se detallarán los métodos de prueba aplicados, los casos de prueba diseñados, los resultados obtenidos y las conclusiones derivadas de estos resultados. Este enfoque integral asegura que el sistema de gestión de reportes de conducta sea robusto, eficiente y confiable.

2. Alcance

El objetivo de las pruebas de integración es verificar la correcta interacción entre los diferentes componentes del sistema de gestión de reportes de conducta. Estas pruebas aseguran que los módulos individuales, previamente validados mediante pruebas unitarias, funcionen conjuntamente de manera coherente y sin errores.

Entorno de Pruebas:

- **Framework de Pruebas Unitarias:** Jest
- **Biblioteca para Pruebas API:** Supertest
- **Pruebas de Navegador:** Selenium WebDriver
- **Navegador:** Firefox
- **Backend:** Node.js
- **Cliente:** Aplicación web en Vue.js

Componentes Probados:

Autenticación de Usuarios:

- Verificación de credenciales de inicio de sesión.
- Validación del proceso de cierre de sesión.

Consulta de Reportes:

- Búsqueda de reportes de conducta por identificador.
- Validación de los resultados de búsqueda.

Generación de Reportes:

- Ingreso de datos del estudiante y motivo del reporte.
- Confirmación de la creación exitosa del reporte.

Criterios de Éxito:

- Los usuarios deben poder iniciar y cerrar sesión correctamente.
- Las consultas deben devolver resultados precisos y relevantes.
- Los reportes deben generarse y almacenarse correctamente en el sistema.
- Las notificaciones y alertas deben mostrarse adecuadamente al usuario.

Descripción de las Pruebas: Las pruebas de integración se ejecutan mediante scripts automatizados utilizando Selenium WebDriver. A continuación, se describen las funciones principales probadas:

- **Login:** Verifica que los usuarios puedan autenticarse correctamente.
- **Consulta:** Valida que las búsquedas de reportes devuelvan los resultados esperados.
- **Generar Reporte:** Asegura que los reportes de conducta se puedan crear y almacenar sin errores.

Estas pruebas se ejecutan en un entorno controlado para garantizar la reproducibilidad y precisión de los resultados. Los detalles de cada prueba, incluyendo los pasos específicos y los resultados obtenidos, se documentan en las secciones siguientes del informe.

3. Herramientas Utilizadas

- **Framework de Pruebas Unitarias:** Jest.
- **Biblioteca para Pruebas API:** Supertest.
- **Pruebas de Navegador:** Selenium WebDriver.

Carretera a la Estación de Rincón Km. 1 C.P. 20670 Pabellón de Arteaga, Aguascalientes

Tel. 465 958-2482 Ext. 104 e-mail: plan_parteaga@tecnm.mx tecnm.mx |

pabellon.tecnm.mx

- **Navegador:** Firefox.
- **Backend:** Node.js.
- **Cliente:** Aplicación web en Vue.js.

4. Metodología

La metodología aplicada en las pruebas de integración del sistema de gestión de reportes de conducta se basa en un enfoque estructurado y sistemático para garantizar la calidad y funcionalidad del sistema. A continuación, se describen los pasos y técnicas empleadas:

Planificación de Pruebas:

- **Definición de Objetivos:** Establecer los objetivos específicos de las pruebas de integración, como la validación de la interacción entre módulos y la detección de defectos en la integración.
- **Selección de Herramientas:** Elegir las herramientas adecuadas para la automatización y ejecución de pruebas, incluyendo Jest, Supertest y Selenium WebDriver.

Diseño de Casos de Prueba:

- **Identificación de Escenarios:** Determinar los escenarios de prueba relevantes que cubran las funcionalidades críticas del sistema, como el inicio de sesión, la consulta de reportes y la generación de reportes.
- **Definición de Datos de Prueba:** Crear conjuntos de datos de prueba que simulen condiciones reales de uso del sistema.

Implementación de Pruebas:

- **Desarrollo de Scripts de Prueba:** Escribir scripts automatizados utilizando Selenium WebDriver para ejecutar las pruebas en el navegador Firefox.
- **Configuración del Entorno:** Configurar el entorno de pruebas, asegurando que todos los componentes del sistema estén disponibles y correctamente configurados.

Ejecución de Pruebas:

- **Ejecución Automatizada:** Ejecutar los scripts de prueba en el entorno configurado, monitorizando la ejecución para detectar cualquier fallo o comportamiento inesperado.
- **Registro de Resultados:** Documentar los resultados de cada prueba, incluyendo los pasos realizados, los resultados obtenidos y cualquier error encontrado.

Análisis de Resultados:

- **Evaluación de Resultados:** Analizar los resultados de las pruebas para identificar patrones de fallos y áreas problemáticas.
- **Reporte de Defectos:** Registrar y priorizar los defectos encontrados, proporcionando detalles suficientes para su resolución.

Revisión y Mejora:

- **Revisión de Pruebas:** Revisar los casos de prueba y los scripts para asegurar su efectividad y cobertura.
- **Mejora Continua:** Implementar mejoras en el proceso de pruebas basado en los hallazgos y retroalimentación obtenida durante la ejecución.

Esta metodología asegura un enfoque riguroso y exhaustivo para las pruebas de integración, contribuyendo a la entrega de un sistema robusto y confiable.

5. Casos de Prueba

5.1. Pruebas Unitarias

5.1. Pruebas Unitarias: Código: Ambas rutas

Variables:

```
import request from 'supertest';
import { app } from '../src/app';
import { describe, it, expect, beforeEach, afterEach } from '@jest/globals';

// Interfaz que define la estructura de un estudiante
interface Student {
  _id?: string; // ID del estudiante
  __v?: number; // Versión del documento
  nombre: string; // Nombre del estudiante
  contraseña: string; // Contraseña del estudiante
  semestre: string; // Semestre en el que está el estudiante
}
```

```
grupo: string; // Grupo al que pertenece el estudiante
usuario: string; // Usuario/ID del estudiante
fotografia?: string; // URL de la fotografía del estudiante
reportes?: string[]; // Array de IDs de reportes asociados al estudiant
cambioContrasena?: boolean; // Indica si se ha cambiado la contraseña
puntos?: number; // Puntos acumulados por el estudiante
fueraSistema?: boolean; // Indica si el estudiante está fuera del sistema
}
// Estructura esperada para un estudiante en las respuestas de la API
const StudentExpected = {
  _id: expect.any(String),
  __v: expect.any(Number),
  nombre: expect.any(String),
  contraseña: expect.any(String),
  semestre: expect.any(String),
  grupo: expect.any(String),
  usuario: expect.any(String),
  fotografia: expect.any(String),
  reportes: expect.any(Array),
  cambioContrasena: expect.any(Boolean),
  puntos: expect.any(Number),
  fueraSistema: expect.any(Boolean)
};
// Datos de un estudiante que se utilizarán para crear un nuevo estudiante en
la API
const studentToCreate: Student = {
  nombre: 'Test',
  contraseña: 'test',
  semestre: "4",
  grupo: 'A',
  usuario: '3112222122',
  fotografia: 'test'
};
// Nueva contraseña para actualizar la contraseña del estudiante
const newPass = '123456';
const students = [
  {
    "_id": "66edaa5b9b9236c115d1591d",
    "nombre": "true Paco",
    "contrasena": "1234",
    "semestre": "3",
    "grupo": "A",
    "usuario": "201050207",
```

```
"fotografia": "https://res.cloudinary.com/dkdq8xhwq/image/authenticated/s-  
-RWNctuzN--/v1729064681/uploads/201050207.jpeg.jpg",  
"reportes": [  
  {  
    "$oid": "66fab7f0e61480cfb2970eba"  
  }  
],  
"puntos": 20,  
"__v": 0,  
"cambioContrasena": true  
},  
{  
  "_id": "66f17e658d1e792430a1e5e1",  
  "nombre": "Ricardo",  
  "contrasena": "",  
  "semestre": "4",  
  "grupo": "C",  
  "usuario": "10103",  
  "fotografia": "https://res.cloudinary.com/dkdq8xhwq/image/authenticated/s-  
-a0N4AkXW--/v1729546421/uploads/10103.jpg",  
  "reportes": [  
    {  
      "$oid": "66f182488d1e792430a1e5f8"  
    }  
  ],  
  "puntos": 198,  
  "__v": 0  
},  
];  
  
// Rutas de la API de estudiantes  
describe('Students API Routes', () => {  
  let token = ''; // Token de autorización  
  let studentId = ''; // ID del estudiante creado  
  // Función para establecer el encabezado de autorización en las solicitudes  
  const auth = () => ({ Authorization: `${token}` });  
  // Antes de todas las pruebas, se obtiene un token de autorización  
  beforeAll(async () => {  
    const response = await request(app)  
      .get('/api/estudiantes/test');  
    token = response.body.token.tokenforTest;  
  });  
  // Prueba para crear un nuevo estudiante  
  it('should return 201 and create a student', async () => {
```

```
const response = await request(app)
  .post('/api/estudiantes')
  .send(studentToCreate)
  .set(auth());
expect(response.body).toEqual(StudentExpected);
expect(response.status).toBe(201);
// Técnica: Partición de equivalencia (validar creación con datos válidos)
});
// Prueba para obtener todos los estudiantes
it('should return more than one student', async () => {
  const response = await request(app)
    .get('/api/estudiantes')
    .set(auth());
  expect(response.status).toBe(200);
  expect(response.body.length).toBeGreaterThan(1);
  // Técnica: Prueba de casos de uso (verificar la obtención de varios
  // estudiantes)
});
// Prueba para obtener un estudiante que no existe
it('should return 404 for non-existent student', async () => {
  const id = '60f3b3b3b3b3b3b3b3b3b3b3';
  const response = await request(app)
    .get(`/api/estudiantes/get/${id}`)
    .set(auth());
  expect(response.status).toBe(404);
  // Técnica: Análisis del valor límite (verificar cómo responde con un ID
  // no válido)
});
// Prueba para obtener un estudiante por su nombre de usuario
it('should return 200 and the student by username', async () => {
  const response = await request(app)
    .get(`/api/estudiantes/username/${studentToCreate.usuario}`)
    .set(auth());
  expect(response.body).toEqual(StudentExpected);
  studentId = response.body._id;
  expect(response.status).toBe(200);
  // Técnica: Partición de equivalencia (verificar obtención con un nombre
  // de usuario válido)
});
// Prueba para obtener un estudiante por su ID
it('should return 200 and the student by ID', async () => {
  const response = await request(app)
    .get(`/api/estudiantes/get/${studentId}`)
```

```
.set(auth());
expect(response.body).toEqual(StudentExpected);
expect(response.status).toBe(200);
// Técnica: Prueba de casos de uso (verificar obtención por ID)
});
// Prueba para actualizar la contraseña de un estudiante
it('should return 200 and update the student password', async () => {
  const response = await request(app)
    .patch(`/api/estudiantes/auth/changePassword/${studentId}`)
    .send({ oldPass: studentToCreate.contrasena, newPass: newPass, change:
true });
  .set(auth());
  expect(response.body).toEqual(StudentExpected);
  expect(response.status).toBe(200);
  // Técnica: Tabla de decisiones (verificar actualización de la contraseña)
});
// Prueba para actualizar el semestre de varios estudiantes
it('should return 200 and update students\' semester', async () => {
  const response = await request(app)
    .post(`/api/estudiantes/semestre`)
    .send({ students: students.map(student => student._id?.toString()) })
    .set(auth());
  expect(response.body.length).toBeGreaterThan(1);
  expect(response.status).toBe(200);
  // Técnica: Prueba de casos de uso (verificar la actualización del
semestre de varios estudiantes)
});
// Prueba para eliminar un estudiante
it('should return 200 and delete the student', async () => {
  const response = await request(app)
    .delete(`/api/estudiantes/${studentId}`)
    .set(auth());
  expect(response.body).toEqual(StudentExpected);
  expect(response.status).toBe(200);
  // Técnica: Partición de equivalencia (verificar eliminación de un
estudiante válido)
});
});
// Estructura esperada para un maestro en las respuestas de la API
const TeacherExpected = {
  __v: expect.any(Number),
  _id: expect.any(String),
  nombre: expect.any(String),
```

```
contrasena: expect.any(String),
usuario: expect.any(String),
fotografia: expect.any(String),
isAdmin: expect.any(Boolean),
cambioContrasena: expect.any(Boolean),
nonAdmin: expect.any(Boolean)
});
// Datos de un maestro que se utilizarán para crear un nuevo maestro en la API
const teacherToCreate = {
  nombre: 'Test',
  contrasena: 'test',
  usuario: '123122224',
  isAdmin: false,
  nonAdmin: true
};
// Descripción de las pruebas para las rutas de la API de maestros
describe('Teachers API Routes', () => {
  let token = ''; // Token de autorización
  let teacherId = ''; // ID del maestro creado
  // Función para establecer el encabezado de autorización en las solicitudes
  const auth = () => ({ Authorization: `${token}` });
  // Antes de todas las pruebas, se obtiene un token de autorización
  beforeAll(async () => {
    const response = await request(app)
      .get('/api/estudiantes/test');
    token = response.body.token?.tokenforTest;
  });
  // Después de todas las pruebas, se eliminan los datos del maestro creado si
  // existe
  afterAll(async () => {
    if (teacherId) {
      await request(app).delete(`/api/maestros/${teacherId}`).set(auth());
    }
  });
  // Prueba para crear un nuevo maestro
  it('should return 201 and create a teacher', async () => {
    const response = await request(app)
      .post('/api/maestros')
      .send(teacherToCreate)
      .set(auth());
    expect(response.status).toBe(201);
    expect(response.body).toEqual(TeacherExpected);
    teacherId = response.body._id; // Almacena el ID del maestro creado
  });
});
```

```
// Técnica: Partición de equivalencia (verificar creación de maestro con
datos válidos)
});
// Prueba para obtener todos los maestros
it('should return more than one teacher', async () => {
  const response = await request(app)
    .get('/api/maestros')
    .set(auth());
  expect(response.status).toBe(200);
  expect(response.body.length).toBeGreaterThan(0); // Verifica que haya al
menos un maestro
  // Técnica: Prueba de casos de uso (verificar obtención de múltiples
maestros)
});
// Prueba para obtener un maestro que no existe
it('should return 404 for a non-existent teacher', async () => {
  const response = await request(app)
    .get('/api/maestros/get/60f3b3b3b3b3b3b3b3b3b3b3b3b3b3b3')
    .set(auth());
  expect(response.status).toBe(404); // Verifica que el estado de la
respuesta sea 404
  // Técnica: Análisis del valor límite (verificar cómo responde con un ID
no válido)
});
// Prueba para obtener un maestro por su nombre de usuario
it('should return 200 and the teacher by username', async () => {
  const response = await request(app)
    .get(`/api/maestros/user/${teacherToCreate.usuario}`)
    .set(auth());
  expect(response.status).toBe(200);
  expect(response.body).toEqual(TeacherExpected); // Verifica que la
respuesta sea la esperada
  // Técnica: Partición de equivalencia (verificar obtención con un nombre
de usuario válido)
});
// Esta prueba verifica que se devuelve un estado 200 y el maestro por ID.
it('should return 200 and the teacher by ID', async () => {
  const response = await request(app)
    .get(`/api/maestros/get/${teacherId}`)
    .set(auth());
  expect(response.status).toBe(200);
  expect(response.body).toEqual(TeacherExpected);
  // Técnica: Prueba de casos de uso (verificar obtención por ID)
```

```
});  
  
// Esta prueba verifica que se devuelve un estado 200 y se actualiza la  
contraseña del maestro.  
it('should return 200 and update the teacher password', async () => {  
  const newPassword = 'newPass123';  
  const response = await request(app)  
    .patch(`/api/maestros/auth/changePassword/${teacherId}`)  
    .send({ oldPass: teacherToCreate.contrasena, newPass: newPassword,  
change: true })  
    .set(auth());  
  expect(response.status).toBe(200);  
  expect(response.body).toEqual(TeacherExpected);  
  // Técnica: Tabla de decisiones (verificar actualización de la contraseña)  
});  
// Esta prueba verifica que se devuelve un estado 200 y se elimina al  
maestro.  
it('should return 200 and delete the teacher', async () => {  
  const response = await request(app)  
    .delete(`/api/maestros/${teacherId}`)  
    .set(auth());  
  expect(response.status).toBe(200);  
  expect(response.body).toEqual(TeacherExpected);  
  teacherId = '';  
  // Técnica: Partición de equivalencia (verificar eliminación de un maestro  
válido)  
});  
});
```

Resultados:

```
PASS tests/app.test.ts (5.676 s)  
Students API Routes  
  ✓ should return 201 and create a student (109 ms)  
  ✓ should return more than one student (23 ms)  
  ✓ should return 404 for non-existent student (7 ms)  
  ✓ should return 200 and the student by username (7 ms)  
  ✓ should return 200 and the student by ID (8 ms)  
  ✓ should return 200 and update the student password (126 ms)  
  ✓ should return 200 and update students' semester (19 ms)  
  ✓ should return 200 and delete the student (7 ms)  
Teachers API Routes  
  ✓ should return 201 and create a teacher (65 ms)  
  ✓ should return more than one teacher (6 ms)  
  ✓ should return 404 for a non-existent teacher (5 ms)  
  ✓ should return 200 and the teacher by username (4 ms)  
  ✓ should return 200 and the teacher by ID (4 ms)  
  ✓ should return 200 and update the teacher password (118 ms)  
  ✓ should return 200 and delete the teacher (5 ms)
```

Carretera a la Estación de Rincón Km. 1 C.P. 20670 Pabellón de Arteaga, Aguascalientes

Tel. 465 958-2482 Ext. 104 e-mail: plan_parteaga@tecnm.mx tecnm.mx |

pabellon.tecnm.mx

Rutas de Estudiantes

Caso	Descripción	Resultado Esperado	Resultado Obtenido
Crear Estudiante	Crear un nuevo estudiante con datos válidos.	Código 201 y datos esperados.	Éxito.
Consultar Todos	Obtener todos los estudiantes registrados.	Código 200 y lista mayor a 1.	Éxito.
Consultar Inexistente	Consultar un estudiante inexistente por ID.	Código 404.	Éxito.
Consultar por usuario	Obtener un estudiante registrado por usuario.	Código 200 y usuario	Éxito.
Consultar por id	Obtener un estudiante registrado por id.	Código 200 y usuario	Éxito.
Cambiar contraseña	Prueba endpoint de cambio de contraseña.	Código 200 y usuario modificado	Éxito.
Aumentar semestre	Se prueba el aumento de semestre en el arreglo de usuarios enviado.	Código 200 y usuarios modificados	Éxito
Borrar estudiante	Se prueba el funcionamiento del endpoint de borrado de estudiantes	Código 200 y un 1 o 0 en caso de error	Éxito

Rutas de Maestros

Caso	Descripción	Resultado Esperado	Resultado Obtenido
Crear Estudiante	Crear un nuevo estudiante con datos válidos.	Código 201 y datos esperados.	Éxito.
Consultar Todos	Obtener todos los estudiantes registrados.	Código 200 y lista mayor a 1.	Éxito.
Consultar Inexistente	Consultar un estudiante inexistente por ID.	Código 404.	Éxito.
Consultar por usuario	Consultar un estudiante existente por usuario.	Código 200 y el usuario.	Éxito.

Carretera a la Estación de Rincón Km. 1 C.P. 20670 Pabellón de Arteaga, Aguascalientes

Tel. 465 958-2482 Ext. 104 e-mail: plan_parteaga@tecnm.mx | tecnm.mx

pabellon.tecnm.mx

Caso	Descripción	Resultado Esperado	Resultado Obtenido
Consultar por id	Consultar un estudiante existente por ID.	Código 200 y el usuario	Éxito.
Cambiar contraseña	Cambiar contraseña de estudiante	Código 201 y el usuario modificado	Éxito
Borrar docente	Borrar docente por Id	Código 201 y un 1 o 0 en caso de error	Éxito

5.2. Pruebas de Integración

5.2. Pruebas de integración: Código: Ambas rutas

Variables:

```
import { Builder, By, Key, until } from 'selenium-webdriver';

// Variables
const url = "http://localhost:5173/";
const loginCredentials = {
  username: '123332',
  password: '1234'
};
const reporteData = {
  nombre: 'Estudiante Prueba',
  razon: 'Test',
  puntos: 1,
  clase: 'Español',
};
const estudianteData = {
  nombre: 'Estudiante Prueba',
  contrasena : '1234',
  semestre: '3',
  grupo: 'A',
  carrera: 'Ingeniería en Sistemas',
  identificador: '1233311'
};
const docenteData = {
  nombre: 'prueba Docente',
  contrasena: '1234',
```

```
    identificador: '3123232',
    isAdmin: true,
  };
  // Función de login sin cerrar sesión
  const loginNoClose = async (driver) => {
    await driver.get(url);
    await driver.wait(until.urlIs(`${url}`), 5000);

    // Envía las credenciales de login
    const username = await driver.findElement(By.id('username'));
    await username.sendKeys(loginCredentials.username);
    const password = await driver.findElement(By.id('password'));
    await password.sendKeys(loginCredentials.password);
    const loginButton = await driver.findElement(By.id('Btnlogin'));
    await loginButton.click();
    await driver.wait(until.urlIs(`${url}consulta`), 5000);
    if (await driver.getCurrentUrl() === `${url}consulta`) {
      console.log('Login successful');
    } else {
      console.log('Login failed');
    }
    return driver;
    // Técnica de prueba de condición múltiple
  };
  // Función de logout
  const logout = async (driver) => {
    await driver.wait(until.elementLocated(By.id('cerrar')), 5000);
    const logoutButton = await driver.findElement(By.id('cerrar'));
    await logoutButton.click();

    await driver.wait(until.urlIs(`${url}`), 5000);
    if (await driver.getCurrentUrl() !== `${url}`) {
      console.log('Logout failed');
    }
    // Técnica de prueba de condición
  };
  // Función de login con cierre de sesión después
  const login = async (driver) => {
    await driver.get(url);
    const username = await driver.findElement(By.id('username'));
    await username.sendKeys(loginCredentials.username);
    const password = await driver.findElement(By.id('password'));
    await password.sendKeys(loginCredentials.password);
```

```
const loginButton = await driver.findElement(By.id('Btnlogin'));
await loginButton.click();
await driver.wait(until.urlIs(`${url}consulta`), 5000);

// Verifica si el login fue exitoso
if (await driver.getCurrentUrl() === `${url}consulta`) {
    console.log('Login successful');
} else {
    console.log('Login failed');
}
await logout(driver); // Cierra sesión después de probar login
// Técnica de prueba de decisión
};

// Función de consulta de estudiante
const consulta = async (driver) => {
    await loginNoClose(driver);
    await driver.wait(until.urlIs(`${url}consulta`), 5000);
    try {
        // Localiza la caja de texto y escribe el nombre
        const textBox = await
driver.wait(until.elementLocated(By.className('identifier')), 5000);
        await textBox.clear();
        await textBox.sendKeys(reporteData.nombre);
        await driver.wait(until.elementLocated(By.css(".results-list")), 5000);
        const resultItem = await driver.wait(
            until.elementLocated(By.css(".results-list .result-item")),
            8000
        );
        await driver.wait(until.elementIsVisible(resultItem), 5000);
        const updatedResultItem = await driver.findElement(By.css(".results-
list .result-item"));
        const resultText = await updatedResultItem.getText();
        if (resultText === "No se encontraron estudiantes.") {
            console.log('.');
        } else if (resultText === `Ricardo (10103)`) {
            console.log('Consulta successful');
        } else {
            console.log('Consulta failed: Resultado inesperado');
        }
    } catch (error) {
        console.error('Error en consulta:', error.message);
    } finally {
```

```
        await logout(driver);
    }
    // Técnica de prueba de ruta
};
// Función para generar un reporte
const generarReporte = async (driver) => {
    await loginNoClose(driver);
    await driver.get(`${url}reporte`);
    await driver.wait(until.urlIs(`${url}reporte`), 5000);
    try {
        // Completa el formulario de reporte
        const nameLabel = await
driver.wait(until.elementLocated(By.id('nombreEstudiante')), 5000);
        await nameLabel.sendKeys(reporteData.nombre);
        const suggestionItems = await
driver.wait(until.elementsLocated(By.className('suggestion-item')), 5000);
        for (let item of suggestionItems) {
            const text = await item.getText();
            if (text === reporteData.nombre) {
                await item.click();
                const razon = await
driver.wait(until.elementLocated(By.id('razonReporte')), 5000);
                await razon.sendKeys(reporteData.razon);
                const puntos = await
driver.wait(until.elementLocated(By.id('puntos')), 5000);
                await puntos.sendKeys(reporteData.puntos);
                const clase = await
driver.wait(until.elementLocated(By.id('clase')), 5000);
                await clase.sendKeys(reporteData.clase);
                const submitButton = await
driver.findElement(By.className('submit-button'));
                await submitButton.click();
                try {
                    await driver.wait(until.alertIsPresent(), 2000);
                    const alert = await driver.switchTo().alert();
                    await alert.dismiss();
                } catch (alertError) {
                    console.log('.');
                }
                try {
                    const toastSuccess = await
driver.wait(until.elementLocated(By.className('v-toast__item--success')),
5000);
```

```
        if (toastSuccess) {
            console.log('Generar reporte successful');
        }
    } catch (successError) {
        console.log('Generar reporte failed: No success or error
toast found');
    }
    break; // Sale del bucle después de generar el reporte
}
}
} catch (error) {
    console.error('Error in generating report:', error.message);
} finally {
    await logout(driver);
}
// Técnica de prueba de casos de uso
};
// Función para añadir estudiante
const añadirEstudiante = async (driver) => {
    await loginNoClose(driver);
    await driver.get(`${url}subir`);
    await driver.wait(until.urlIs(`${url}subir`), 5000);
    try {
        // Rellena el formulario de estudiante
        const nombre = await driver.wait(until.elementLocated(By.id('nombre')),
5000);
        await nombre.sendKeys(estudianteData.nombre);
        const apellido = await
driver.wait(until.elementLocated(By.id('contrasena')), 5000);
        await apellido.sendKeys(estudianteData.contrasena);
        const grado = await
driver.wait(until.elementLocated(By.id('semestre')), 5000);
        await grado.sendKeys(estudianteData.semestre);
        const grupo = await driver.wait(until.elementLocated(By.id('grupo')),
5000);
        await grupo.sendKeys(estudianteData.grupo);

        const identificador = await
driver.wait(until.elementLocated(By.id('identificador')), 5000);
        await identificador.sendKeys(estudianteData.identificador);
        const button = await driver.wait(until.elementLocated(By.id('add-
btn')), 5000);
        await button.click();
    } catch (error) {
        console.error('Error adding student:', error.message);
    }
};
```

```
// Manejo de alertas y toasts
try {
  await driver.wait(until.alertIsPresent(), 2000);
  const alert = await driver.switchTo().alert();
  await alert.dismiss();
} catch (alertError) {
  console.log('.');
}
try {
  const toastSuccess = await
driver.wait(until.elementLocated(By.className('v-toast__item--success')),
5000);
  if (toastSuccess) {
    console.log('Estudiante añadido successful');
  }
}
catch (successError) {
  console.log('Estudiante añadido failed: No success or error toast
found');
}
} catch (error) {
  console.error('Error in añadir estudiante:', error.message);
} finally {
  await logout(driver);
}
// Técnica de pruebaa de particion de equivalencias
};
// Función para ejecutar todas las pruebas
const runTests = async () => {
  let driver = await new Builder().forBrowser('firefox').build();
  try {
    await login(driver);
    await consulta(driver);
    await añadirEstudiante(driver);
    await generarReporte(driver);
    await modificarEstudiante(driver);
    await añadirDocente(driver);
    await cambiarContrasenaDocente(driver);
    await cambiarContrasenaEstudiante(driver);
  } catch (error) {
    console.error('Error during tests:', error.message);
  } finally {
    await driver.quit();
  }
}
```

```
}  
};  
runTests();
```

Resultados:

```
Login successful  
Login successful  
.  
Login successful  
.  
Estudiante añadido successful  
Login successful  
.  
Generar reporte successful  
Login successful  
.  
Modificar estudiante successful  
Login successful  
.  
Docente añadido successful  
.  
Cambiar contraseña successful
```

Flujo de Usuarios

Caso	Descripción	Pasos	Resultado Esperado	Resultado Obtenido
Login	Autenticarse con credenciales válidas.	Introducir usuario y contraseña, enviar.	Redirección a "/consulta".	Éxito.
Consulta	Consultar un estudiante inexistente.	Buscar "Test".	"No se encontraron..."	Éxito.
Reportes	Generar reporte filtrando por fecha inicial.	Seleccionar fechas, generar reporte.	Visualización del reporte.	Éxito.
Logout	Cerrar sesión correctamente.	Clic en "cerrar".	Redirección a página inicial.	Éxito.
Añadir estudiante	Se agrega un estudiante	Introducir los datos de un estudiante y enviarlos	El estudiante se crea con éxito	Éxito
Modificar estudiante	Se modifican los campos de un estudiante	Buscar un estudiante y modificar sus datos	El estudiante se modifica con éxito	Éxito
Añadir docente	Se agrega un docente	Introducir los datos de un docente y enviarlos	El docente se crea con éxito	Éxito

Carretera a la Estación de Rincón Km. 1 C.P. 20670 Pabellón de Arteaga, Aguascalientes

Tel. 465 958-2482 Ext. 104 e-mail: plan_parteaga@tecnm.mx tecnm.mx |

pabellon.tecnm.mx



Caso	Descripción	Pasos	Resultado Esperado	Resultado Obtenido
Cambiar contraseña docente	Se cambia la contraseña de un usuario docente	Ingresar con un docente y cambiar su contraseña	La contraseña se cambia con éxito	Éxito
Cambiar contraseña estudiante	Se cambia la contraseña de un usuario estudiante	Ingresar con un estudiante y cambiar su contraseña	La contraseña se cambia con éxito	Éxito

5.3. Pruebas de Sistema:

Se llevaron a cabo pruebas de aceptación para validar el sistema completo en un entorno real, permitiendo que algunos estudiantes accedieran al sistema a través de la red del Tecnológico. El objetivo fue garantizar que las funcionalidades principales cumplieran con los requisitos establecidos y que el sistema pudiera operar correctamente en condiciones de uso real.

```

Login successful
Login successful
.
Login successful
.
Estudiante añadido successful
Login successful
.
Generar reporte successful
Login successful
.
Modificar estudiante successful
Login successful
.
Docente añadido successful
.
Cambiar contraseña successful

```

5.3. Resultados Pruebas de Sistema:

Caso	Descripción	Pasos	Resultado Esperado	Resultado Obtenido
Login	Autenticarse con credenciales válidas.	Introducir usuario y contraseña, enviar.	Redirección a "/consulta".	Éxito.
Consulta	Consultar un estudiante	Buscar "Test".	"No se encontraron..."	Éxito.

Caso	Descripción	Pasos	Resultado Esperado	Resultado Obtenido
	inexistente.			
Reportes	Generar reporte filtrando por fecha inicial.	Seleccionar fechas, generar reporte.	Visualización del reporte.	Éxito.
Logout	Cerrar sesión correctamente.	Clic en "cerrar".	Redirección a página inicial.	Éxito.
Añadir estudiante	Se agrega un estudiante	Introducir los datos de un estudiante y enviarlos	El estudiante se crea con éxito	Éxito
Modificar estudiante	Se modifican los campos de un estudiante	Buscar un estudiante y modificar sus datos	El estudiante se modifica con éxito	Éxito
Añadir docente	Se agrega un docente	Introducir los datos de un docente y enviarlos	El docente se crea con éxito	Éxito
Cambiar contraseña docente	Se cambia la contraseña de un usuario docente	Ingresa con un docente y cambiar su contraseña	La contraseña se cambia con éxito	Éxito
Cambiar contraseña estudiante	Se cambia la contraseña de un usuario estudiante	Ingresa con un estudiante y cambiar su contraseña	La contraseña se cambia con éxito	Éxito

Observaciones: Al realizar las pruebas de integración en un entorno de producción, redirigiendo a un servidor antes de consumir el servicio, las pruebas de integración resultaron exitosas, solo el tiempo de espera aumentó un 527% ya que pasó de un promedio de 87ms a 545 ms promedio

Observaciones Adicionales: Pruebas de Estrés

Durante las pruebas de aceptación, se evaluaron indirectamente las capacidades del sistema bajo condiciones de estrés. Se utilizó el sistema para probar diferentes partes simultáneamente, simulando un entorno con múltiples solicitudes concurrentes.

Resultados:

Carretera a la Estación de Rincón Km. 1 C.P. 20670 Pabellón de Arteaga, Aguascalientes

Tel. 465 958-2482 Ext. 104 e-mail: plan_parteaga@tecnm.mx tecnm.mx |

pabellon.tecnm.mx



El sistema manejó sin problemas 14 solicitudes concurrentes enviadas a intervalos de 1-3 segundos.

Las solicitudes incluían operaciones críticas, como crear y consultar objetos en tiempo real.

Conclusión:

El sistema demostró estabilidad y capacidad para manejar un nivel moderado de tráfico concurrente, validando su rendimiento en escenarios de uso simultáneo.

5.4. Pruebas de Aceptación:

Se llevaron a cabo pruebas de aceptación para validar el sistema completo en un entorno real, permitiendo que algunos estudiantes accedieran al sistema a través de la red del Tecnológico. El objetivo fue garantizar que las funcionalidades principales cumplieran con los requisitos establecidos y que el sistema pudiera operar correctamente en condiciones de uso real.

5.4. Pruebas de Aceptación:

Código: Ambas ruta:

Variables:Código probado:

Ambas rutas críticas del sistema fueron evaluadas en términos de integración.

Variables:

Flujos completos para estudiantes y docentes.

Operaciones CRUD (Crear, Consultar, Actualizar, Eliminar).

Generación de reportes.

Modificación de contraseñas.

6. Resultados

Se ejecutaron 15 casos de prueba, todos con una tasa de éxito del 100%.

Pruebas de Integración:

Se realizaron flujos completos exitosos que incluyeron:

- Creación, consulta, actualización y eliminación de estudiantes y docentes.
- Generación de reportes.
- Modificación de contraseñas.

Observaciones:

Todos los resultados esperados fueron cumplidos sin fallos. Sin embargo, se detectó un mensaje de advertencia en Jest sobre operaciones asincrónicas no detenidas al finalizar las pruebas. Esto no afectó los resultados, pero se recomienda investigar usando la opción `--detectOpenHandles` para evitar problemas en entornos de producción.

7. Conclusiones

El sistema cumple con las funcionalidades clave según los casos de prueba definidos. Las pruebas reflejan:

- La correcta interacción entre los módulos de estudiantes, docentes y reportes.
- La integridad en la creación, consulta, modificación y eliminación de registros.
- La estabilidad de los flujos principales de uso del sistema.
- **Pruebas de aceptación:** En el entorno de aceptación, al abrir el sistema a la red del Tecnológico, se permitió que estudiantes reales interactuaran con el sistema. Este enfoque confirmó la usabilidad y funcionalidad del sistema en un entorno real, con resultados positivos.

Además de que el sistema demuestra ser funcional en entornos fuera de prueba, es bastante gratificante el poder ver que el promedio de tiempo de respuesta en este entorno fue 545 ms.